Shafar Foundation presents

Battle of Brains 2026

Editorial

Date: August 17, 2026

Problem Summary

#	**Problem Name**	**Rating**	**Tags**
A	The Forbidden Symphony	2100	String, Aho-Corasick, Matrix Exponentiation, DP
B	Stock Market Crash	1400	Game Theory, Bitmask
C	Hello, Contestants!	800	Implementation, Ad-hoc
D	Rafat's Business (Easy Version)	1000	Greedy, Implementation
E	Rafat's Business (Hard Version)	1600	Dynamic Programming, Knapsack
F	All in one Gun	1200	Greedy, Math
G	Elite Network Routing	1600	Graph Theory, Binary Search
H	The Final Decision	800	Ad-hoc
I	Triple Key Authentication	1800	Number Theory, Mobius Inversion, Combinatorics
J	Festival Lights	1200	Math

A. The Forbidden Symphony

Editorial

Solution Logic

This problem requires a two-step approach using **Aho-Corasick Automaton** and **Matrix Exponentiation**.

Step 1: Aho-Corasick Automaton (Building the State Machine)

To track N strings simultaneously, we build a Trie (Prefix Tree). However, a simple Trie is not enough because if we fail to match a string, we need to know which other string we might be partially matching (similar to the KMP algorithm). The Aho-Corasick algorithm solves this by creating a `fail` link for each node.

- We treat each node of the Trie as a "State" (represented by an integer).
- If reaching a node completes any forbidden string, we mark that node as `bad`.
- Furthermore, if a node's `fail` link points to a `bad` node, the current node must also be marked as `bad` (because a suffix of the current string is forbidden).

Step 2: Matrix Exponentiation (Handling Large Length)

The value of L can be up to 10^{18}, making it impossible to solve using a standard DP loop.

- The total number of nodes (states) in the Trie will be at most 100 (since the sum of lengths of all forbidden strings is at most 100).
- We construct a square matrix T of size $\approx 100 \times 100$. We set $T[i][j] = 1$ if we can transition from state i to state j by appending a single character, provided that neither i nor j is a `bad` state.
- We need to find the total number of ways to reach any valid state starting from the root (state 0) in exactly L steps.
- According to matrix properties, if we multiply the matrix T by itself L times (i.e., calculate T^L), each cell in the resulting matrix represents the number of ways to transition in exactly L steps. We can efficiently compute T^L in $O(M^3 \log L)$ time using **Matrix Exponentiation**.

Time Complexity

Building the Aho-Corasick Automaton takes $O(\Sigma \times \text{sum of lengths}) = O(26 \times 100)$ time.
Matrix Exponentiation takes $O(M^3 \log L)$ time, where M is the number of valid states ($M \leq 100$).
The total time complexity is $\mathcal{O}(M^3 \log L)$, which easily passes within the time limit.

Source Code (C++)

```
#include <bits/stdc++.h>
using namespace std;

const int MOD = 1e9 + 7;
```

```
struct Matrix {
    int n;
    vector<vector<long long>> mat;

    Matrix(int n) : n(n), mat(n, vector<long long>(n, 0)) {}

    Matrix operator*(const Matrix &other) const {
        Matrix res(n);
        for(int i = 0; i < n; i++) {
            for(int k = 0; k < n; k++) {
                if(mat[i][k] == 0) continue;
                for(int j = 0; j < n; j++) {
                    res.mat[i][j] = (res.mat[i][j] + mat[i][k] * other.
                        mat[k][j]) % MOD;
                }
            }
        }
        return res;
    }
};

Matrix power(Matrix a, long long b) {
    Matrix res(a.n);
    for(int i = 0; i < a.n; i++) res.mat[i][i] = 1;
    while(b > 0) {
        if(b & 1) res = res * a;
        a = a * a;
        b >>= 1;
    }
    return res;
}

int trie_tree[105][26];
int fail[105];
bool bad[105];
int node_cnt = 0;

void insert_str(string s) {
    int u = 0;
    for(char c : s) {
        int v = c - 'a';
        if(!trie_tree[u][v]) {
            trie_tree[u][v] = ++node_cnt;
        }
        u = trie_tree[u][v];
    }
    bad[u] = true;
}

void build_ac() {
    queue<int> q;
    for(int i = 0; i < 26; i++) {
        if(trie_tree[0][i]) {
            q.push(trie_tree[0][i]);
        }
    }
    while(!q.empty()) {
        int u = q.front();
```

```
        q.pop();

        bad[u] |= bad[fail[u]];

        for(int i = 0; i < 26; i++) {
            if(trie_tree[u][i]) {
                fail[trie_tree[u][i]] = trie_tree[fail[u]][i];
                q.push(trie_tree[u][i]);
            } else {
                trie_tree[u][i] = trie_tree[fail[u]][i];
            }
        }
    }
}

int main() {
    ios_base::sync_with_stdio(false);
    cin.tie(NULL);

    int n;
    long long l;
    if(!(cin >> n >> l)) return 0;

    for(int i = 0; i < n; i++) {
        string s;
        cin >> s;
        insert_str(s);
    }

    build_ac();

    int sz = node_cnt + 1;
    Matrix T(sz);

    for(int i = 0; i < sz; i++) {
        if(bad[i]) continue;
        for(int j = 0; j < 26; j++) {
            int nxt = trie_tree[i][j];
            if(!bad[nxt]) {
                T.mat[i][nxt]++;
            }
        }
    }

    T = power(T, l);

    long long ans = 0;
    for(int i = 0; i < sz; i++) {
        ans = (ans + T.mat[0][i]) % MOD;
    }

    cout << ans << "\n";

    return 0;
}
```

B. Stock Market Crash

Editorial

Solution Logic

At first glance, this might seem like a very difficult bitwise problem, but it is actually a variation of the classical **Nim Game**.

When you perform a `Bitwise AND` operation on a number with any other number, the 1 bits in its binary representation can turn into 0, but a 0 bit can never turn into a 1. Since the condition states that the new price must be strictly less than the previous price, it implies that at least one (or more) 1 bits must be turned into 0 during a valid move.

In other words, the number of **Set Bits (number of 1s)** in the binary representation of each share price A_i acts exactly like a pile of stones in the classic Nim game. In each move, a player can remove one or more 1 bits (stones) from exactly one pile (share price).

Therefore, we just need to count the number of set bits for each A_i and calculate their **XOR Sum**.

- If the XOR Sum is strictly greater than 0, Alice (the first player) will win because she can make a move that leaves the XOR sum as 0 for Bob.
- If the XOR Sum is exactly 0, Bob will win.

Time Complexity

For each number A_i, we can count its set bits in $\mathcal{O}(1)$ time using built-in functions like `__builtin_popcountll(a)`.
Thus, for each test case, the time complexity is $\mathcal{O}(N)$.
Across all test cases, the total time complexity will be $\mathcal{O}(\sum N)$, which fits perfectly within the time limit. Space complexity is $\mathcal{O}(1)$.

Source Code (C++)

```cpp
#include <bits/stdc++.h>
using namespace std;

void solve() {
    int n;
    cin >> n;

    long long xor_sum = 0;

    for (int i = 0; i < n; i++) {
        long long a;
        cin >> a;
        long long set_bits = __builtin_popcountll(a);
        xor_sum ^= set_bits;
    }

    if (xor_sum > 0) {
        cout << "Alice\n";
    } else {
        cout << "Bob\n";
    }
}
```

```
int main() {
    ios_base::sync_with_stdio(false);
    cin.tie(NULL);

    int t;
    if (cin >> t) {
        while (t--) {
            solve();
        }
    }

    return 0;
}
```

C. Hello, Contestants!

Editorial

Solution Logic

This is a very basic problem that requires simply printing a predefined set of strings. No input is taken.

Time Complexity

The time complexity is $\mathcal{O}(1)$ since it only involves printing a constant string. The space complexity is also $\mathcal{O}(1)$.

Source Code (C++)

```
#include <iostream>

using namespace std;

int main() {
    ios_base::sync_with_stdio(false);
    cin.tie(NULL);
    cout << "Welcome to Battle of Brains 2026!\nGood luck, contestants
       !\n";
    return 0;
}
```

D. Rafat's Business (Easy Version)

Editorial

Solution Logic

Observations:
The problem states that each item i has a quantity a_i which is strictly either 1 or 2. If Rafat chooses to buy the i-th item, he must buy all a_i units, and the total cost for that choice will be $a_i \times b_i$. He wants to select a subset of items such that the sum of their quantities is exactly k, and their total cost is minimized.

Because $a_i \in \{1, 2\}$, we can divide all items into two groups:

1. Items with quantity 1.
2. Items with quantity 2.

Let's calculate the total cost for each item and push them into two separate arrays `ones` and `twos`.

- For $a_i = 1$, the cost is $1 \times b_i = b_i$.
- For $a_i = 2$, the cost is $2 \times b_i$.

Solution:
Since we want to minimize the cost, if we decide to pick exactly x items from the `twos` group, we should obviously pick the x items with the smallest costs. Similarly, we will need exactly $y = k - 2x$ items from the `ones` group, and we should pick the y items with the smallest costs.
So, the optimal approach is:

1. Sort the `ones` array in ascending order.
2. Sort the `twos` array in ascending order.
3. Compute the prefix sums for both arrays to quickly get the sum of the first x elements in $\mathcal{O}(1)$ time.
4. Iterate over all possible number of items we can pick from the `twos` group. Let this be i ($0 \leq i \leq$ size of twos).
5. For a given i, the number of items needed from the `ones` group is $rem = k - 2 \times i$.
6. If $rem \geq 0$ and $rem \leq$ size of ones, this is a valid combination. The minimum cost for this combination is the sum of the first i elements in `twos` plus the sum of the first rem elements in `ones`.
7. Keep track of the minimum cost across all valid combinations.

If no valid combination is found, the answer is `-1`.

Time Complexity

Sorting the `ones` and `twos` arrays takes $\mathcal{O}(N \log N)$ time.
Iterating over the choices takes $\mathcal{O}(N)$ time.
Thus, the total time complexity per test case is $\mathcal{O}(N \log N)$.
The sum of N over all test cases is bounded by $2 \cdot 10^5$, ensuring the solution runs well within the time limit. Space complexity is $\mathcal{O}(N)$ for storing the arrays and their prefix sums.

Source Code (C++)

```
#include<bits/stdc++.h>
using namespace std;

void solve() {
    int n;
    long long k;
    cin >> n >> k;
    vector<long long> a(n), b(n);
    for (int i = 0; i < n; i++) cin >> a[i];
    for (int i = 0; i < n; i++) cin >> b[i];

    vector<long long> ones, twos;
    for (int i = 0; i < n; i++) {
        if (a[i] == 1) ones.push_back(b[i]);
        else twos.push_back(2LL * b[i]);
    }

    sort(ones.begin(), ones.end());
    sort(twos.begin(), twos.end());

    vector<long long> pref1(ones.size() + 1, 0);
    for (size_t i = 0; i < ones.size(); i++) pref1[i + 1] = pref1[i] +
       ones[i];

    vector<long long> pref2(twos.size() + 1, 0);
    for (size_t i = 0; i < twos.size(); i++) pref2[i + 1] = pref2[i] +
       twos[i];

    long long ans = -1;

    for (size_t i = 0; i <= twos.size(); i++) {
        long long rem = k - 2 * i;
        if (rem >= 0 && rem <= ones.size()) {
            long long cost = pref2[i] + pref1[rem];
            if (ans == -1 || cost < ans) ans = cost;
        }
    }
    cout << ans << "\n";
}

int main() {
    ios_base::sync_with_stdio(false);
    cin.tie(NULL);
    int t;
    cin >> t;
    while (t--) {
        solve();
    }
    return 0;
}
```

E. Rafat's Business (Hard Version)

Editorial

Solution Logic

Observations:
In this version, the quantity a_i of an item is no longer restricted to just 1 or 2. It can be any integer up to 100. If Rafat chooses to buy the i-th item, he must buy all a_i units, and the total cost for that choice will be $a_i \times b_i$. He wants to select a subset of items such that the sum of their quantities is exactly k, and their total cost is minimized.

This is a classic variation of the 0/1 Knapsack Problem. Instead of maximizing the value for a given capacity, we want to **minimize the cost** for an exact target capacity k.

Solution:
Let $dp[j]$ be the minimum cost to exactly buy j items.

Initially, we set:

- $dp[0] = 0$ (cost to buy 0 items is 0)
- $dp[j] = \infty$ for all $j \in [1, k]$ (initially impossible to form other capacities)

We iterate through each item i from 1 to n:
For the i-th item, its "weight" is $w = a_i$ (the quantity), and its "value" (or cost) is $v = a_i \times b_i$.

Since we can pick each item at most once (0/1 Knapsack), we iterate j backwards from k down to w:
If $dp[j - w]$ is not ∞, it means we can reach a capacity of $j - w$. By taking the i-th item, we can now reach a capacity of j. The new cost would be $dp[j - w] + v$. We update $dp[j]$ if this new cost is strictly less than the current value of $dp[j]$.

Transition:

$$dp[j] = \min(dp[j], dp[j - w] + v)$$

After processing all n items, the answer will be $dp[k]$. If $dp[k]$ is still ∞, it means we cannot form exactly k items, so we output `-1`.

Time Complexity

The DP table has size $k + 1$.
For each of the n items, we iterate at most k times.
Thus, the time complexity per testcase is $\mathcal{O}(n \times k)$.
Given $n \leq 1000$, $\sum n \leq 5000$ and $k \leq 10^4$, the total operations across all test cases is roughly around 5×10^7, which will easily pass within the time limit.
Space complexity is $\mathcal{O}(k)$ for the DP array.

Source Code (C++)

```
#include <bits/stdc++.h>
using namespace std;

const long long INF = 1e18;

void solve() {
    int n, k;
    cin >> n >> k;
    vector<int> a(n);
```

```
    vector<long long> b(n);

    for (int i = 0; i < n; ++i) cin >> a[i];
    for (int i = 0; i < n; ++i) cin >> b[i];

    vector<long long> dp(k + 1, INF);
    dp[0] = 0;

    for (int i = 0; i < n; ++i) {
        int w = a[i];
        long long v = 1LL * a[i] * b[i];

        for (int j = k; j >= w; --j) {
            if (dp[j - w] != INF) {
                dp[j] = min(dp[j], dp[j - w] + v);
            }
        }
    }

    if (dp[k] == INF) {
        cout << -1 << "\n";
    } else {
        cout << dp[k] << "\n";
    }
}

int main() {
    ios_base::sync_with_stdio(false);
    cin.tie(NULL);
    int t;
    cin >> t;
    while (t--) {
        solve();
    }
    return 0;
}
```

F. All in one Gun

Editorial

Solution Logic

To minimize the time to kill the enemy, we want to maximize the damage dealt in the shortest amount of time. Since we must reload after every n bullets, the total damage done by a full magazine is constant, regardless of any swaps. Let this sum be $S = \sum_{i=1}^{n} a_i$.

If the fight ends after exactly P bullets of the final magazine ($1 \leq P \leq n$), the time taken is $C \times (n + k) + P$, where C is the number of full magazines fired. The total damage dealt is $C \times S + M_P$, where M_P is the maximum possible sum of the first P bullets after at most one swap.

To minimize the time, we should iterate over all possible values of $P \in [1, n]$. For a fixed P, we want to maximize M_P to minimize C.

Maximizing M_P

To maximize the sum of the first P elements using at most one swap, we should:

1. Find the minimum element in the first P bullets, let's call it `min_pref`.
2. Find the maximum element in the remaining $n - P$ bullets (i.e., from index $P + 1$ to n), let's call it `max_suff`.
3. If `max_suff > min_pref`, we swap them. Otherwise, we don't swap.

Thus, $M_P = \text{pref\_sum}[P] + \max(0LL, \text{max\_suff} - \text{min\_pref})$.

Calculating the Time

- The remaining health to be dealt by full magazines is $\max(0, h - M_P)$.
- The number of full magazines required is $C = \lceil \frac{\max(0LL, h - M_P)}{S} \rceil$.
- The total time is $C \times (n + k) + P$.

We compute this time for all $1 \leq P \leq n$ and take the minimum.

Important Note

Because h can be up to 10^{18}, C can also be up to 10^{18}. Then $C \times (n + k)$ could exceed the maximum value of a 64-bit signed integer (`long long`). We must use `unsigned long long` to avoid overflow during this final multiplication, as the maximum possible answer for the given constraints fits inside a 64-bit unsigned integer but might overflow a signed one.

Complexity

- **Time Complexity:** $\mathcal{O}(n)$ per test case. We can precompute the maximum suffix array in $\mathcal{O}(n)$ and maintain the prefix sum and minimum element on the fly. Overall time complexity is $\mathcal{O}(\sum n)$.
- **Space/Memory Complexity:** $\mathcal{O}(n)$ to store the array and the maximum suffix array.

Source Code (C++)

```cpp
#include <iostream>
#include <vector>
#include <algorithm>

using namespace std;

void solve() {
    int n;
    unsigned long long h, k;
    cin >> n >> h >> k;

    vector<long long> a(n);
    unsigned long long S = 0;
    for (int i = 0; i < n; ++i) {
        cin >> a[i];
        S += a[i];
    }

    vector<long long> max_suff(n + 1, 0);
    for (int i = n - 1; i >= 0; --i) {
        max_suff[i] = max(max_suff[i + 1], a[i]);
    }

    long long min_pref = a[0];
    long long pref = 0;
    unsigned long long ans = -1;

    for (int P = 1; P <= n; ++P) {
        pref += a[P - 1];
        min_pref = min(min_pref, a[P - 1]);

        long long M_P = pref;
        if (P < n && max_suff[P] > min_pref) {
            M_P += max_suff[P] - min_pref;
        }

        unsigned long long C = 0;
        if (h > (unsigned long long)M_P) {
            unsigned long long rem = h - M_P;
            C = (rem + S - 1) / S;
        }

        unsigned long long time = C * (n + k) + P;
        if (time < ans) {
            ans = time;
        }
    }

    cout << ans << "\n";
}

int main() {
    ios_base::sync_with_stdio(false);
    cin.tie(NULL);

    int t;
    if (cin >> t) {
```

```
        while (t--) {
            solve();
        }
    }
    return 0;
}
```

G. Elite Network Routing

Editorial

Solution Logic

Observations:
We need to find a path from server 1 to server N such that the maximum edge weight on the path is minimized. This is a variation of the "Minimax Path" problem. The possible answers lie in the range $[0, 10^9]$.

Solution:
We can use **Binary Search on the answer**.
The range for our binary search will be `low = 0` and `high = 10^9` (or the maximum weight present in the graph).

For a given `mid`, we will run a standard graph traversal algorithm like BFS or DFS starting from server 1.
During the traversal, we will only consider edges with a weight **less than or equal to** `mid`.
If we can reach server N using only these edges, it means it is possible to reach the destination with a maximum risk of `mid` (or possibly lower). So we update our possible answer and search for a smaller risk by setting `high = mid - 1`.
If we cannot reach server N, we must increase our allowed risk limit, so we set `low = mid + 1`.

This approach will efficiently narrow down and find the exact minimum possible maximum risk.

Time Complexity

The binary search takes $\mathcal{O}(\log(\text{max\_w}))$ iterations.
In each iteration, the BFS or DFS visits at most N vertices and M edges, which takes $\mathcal{O}(N+M)$ time.
Total Time Complexity: $\mathcal{O}((N + M)\log(\text{max\_w}))$. Given $N, M \leq 2 \cdot 10^5$, this fits very comfortably within standard time limits. Space complexity is $\mathcal{O}(N + M)$ to store the graph in an adjacency list.

Source Code (C++)

```
#include <bits/stdc++.h>
using namespace std;

struct Edge {
    int to;
    long long weight;
};

vector<vector<Edge>> adj;
int n, m;

bool isValid(long long max_risk) {
    vector<bool> visited(n + 1, false);
    queue<int> q;
    q.push(1);
    visited[1] = true;
    while (!q.empty()) {
        int u = q.front();
        q.pop();
        if (u == n) return true;
```

```
        for (Edge& edge : adj[u]) {
            if (!visited[edge.to] && edge.weight <= max_risk) {
                visited[edge.to] = true;
                q.push(edge.to);
            }
        }
    }
    return visited[n];
}

int main() {
    ios_base::sync_with_stdio(false);
    cin.tie(NULL);

    if (!(cin >> n >> m)) return 0;

    adj.resize(n + 1);
    long long max_w = 0;

    for (int i = 0; i < m; i++) {
        int u, v;
        long long w;
        cin >> u >> v >> w;
        adj[u].push_back({v, w});
        adj[v].push_back({u, w});
        max_w = max(max_w, w);
    }

    long long low = 0, high = max_w;
    long long ans = -1;

    while (low <= high) {
        long long mid = low + (high - low) / 2;
        if (isValid(mid)) {
            ans = mid;
            high = mid - 1;
        } else {
            low = mid + 1;
        }
    }

    cout << ans << "\n";

    return 0;
}
```

H. The Final Decision

Editorial

Solution Logic

This problem tests basic arithmetic operations and condition checking based on the problem statement. The problem asks us to perform one of two operations based on whether the two given integers A and B are equal or not.

- If $A = B$, we need to calculate and print $A \times B$.
- If $A \neq B$, we need to calculate and print $A + B$.

Complexity

- **Time Complexity:** $\mathcal{O}(1)$, because we only perform a few basic comparisons and arithmetic operations.
- **Space/Memory Complexity:** $\mathcal{O}(1)$, because we only use a few variables to store the input integers and the result.

Source Code (C++)

```
#include <iostream>

using namespace std;

int main() {
    long long a, b;
    cin >> a >> b;

    if (a == b) {
        cout << a * b;
    } else {
        cout << a + b;
    }

    return 0;
}
```

I. Triple Key Authentication

Editorial

Solution Logic

This problem can be efficiently solved using **Möbius Inversion** and the **Harmonic Lemma** (also known as the Block Trick).

1. The total number of triplets of any three numbers from 1 to N is simply N^3.
2. If we want to find the number of triplets whose GCD is a multiple of d, the count will be $\lfloor \frac{N}{d} \rfloor^3$.
3. According to Möbius Inversion, the formula to find the exact number of triplets whose GCD is exactly 1 is:
$$ans = \sum_{i=1}^{N} \mu(i) \times \lfloor \frac{N}{i} \rfloor^3$$
4. Since $T = 10^4$ and $N = 10^6$, running an $\mathcal{O}(N)$ loop for each test case will result in a Time Limit Exceeded (TLE) verdict.
5. To optimize this, we can precompute the prefix sums of $\mu(i)$ and use **Square Root Decomposition on Division** (the Block Trick). Since $\lfloor \frac{N}{i} \rfloor$ takes at most $2\sqrt{N}$ distinct values, we can group together ranges of i where $\lfloor \frac{N}{i} \rfloor$ remains the same.
6. As a result, the answer for each query can be computed in $\mathcal{O}(\sqrt{N})$ time.

Time Complexity

- **Sieve and Precomputation:** We use a sieve to compute the Möbius function $\mu(i)$ and its prefix sums up to 10^6, which takes $\mathcal{O}(N)$ time.
- **Query Processing:** Using the block trick, each of the T queries takes $\mathcal{O}(\sqrt{N})$ time.
- **Overall Time Complexity:** $\mathcal{O}(N + T\sqrt{N})$. With $N = 10^6$ and $T = 10^4$, this translates to around $10^6 + 10^4 \times 10^3 = 1.1 \times 10^7$ operations, which will easily pass within the standard time limits.

Source Code (C++)

```
#include <bits/stdc++.h>
using namespace std;

const int MAX = 1e6 + 5;
const long long MOD = 1e9 + 7;

int mu[MAX];
bool is_prime[MAX];
vector<int> primes;
long long pref_mu[MAX];

void sieve() {
    fill(is_prime + 2, is_prime + MAX, true);
```

```
    mu[1] = 1;

    for (int i = 2; i < MAX; i++) {
        if (is_prime[i]) {
            primes.push_back(i);
            mu[i] = -1;
        }
        for (int p : primes) {
            if (i * p >= MAX) break;
            is_prime[i * p] = false;
            if (i % p == 0) {
                mu[i * p] = 0;
                break;
            } else {
                mu[i * p] = mu[i] * -1;
            }
        }
    }

    pref_mu[0] = 0;
    for (int i = 1; i < MAX; i++) {
        pref_mu[i] = (pref_mu[i - 1] + mu[i] % MOD + MOD) % MOD;
    }
}

void solve() {
    long long n;
    cin >> n;

    long long ans = 0;

    for (long long l = 1, r; l <= n; l = r + 1) {
        long long k = n / l;
        r = n / k;

        long long sum_mu = (pref_mu[r] - pref_mu[l - 1] + MOD) % MOD;
        long long ways = (k % MOD) * (k % MOD) % MOD * (k % MOD) % MOD;

        ans = (ans + sum_mu * ways) % MOD;
    }

    cout << ans << "\n";
}

int main() {
    ios_base::sync_with_stdio(false);
    cin.tie(NULL);

    sieve();

    int t;
    if (cin >> t) {
        while (t--) {
            solve();
        }
    }

    return 0;
```

```
}
```

J. Festival Lights

Editorial

Solution Logic

This problem is an application of the Inclusion-Exclusion Principle.

We need to calculate the number of days each person visits. Let V_A, V_B, and V_C be the sets of days Alice, Bob, and Carol visit, respectively.

- Alice visits every a days. The number of days Alice visits in the first m days is $N_A = \lfloor \frac{m}{a} \rfloor$.
- Bob visits every b days. The number of days Bob visits is $N_B = \lfloor \frac{m}{b} \rfloor$.
- Carol visits every c days. The number of days Carol visits is $N_C = \lfloor \frac{m}{c} \rfloor$.

To find out how many days multiple people visit together, we compute the least common multiple (LCM) of their visitation intervals.

- The number of days Alice and Bob visit together is $N_{AB} = \lfloor \frac{m}{\text{lcm}(a,b)} \rfloor$.
- The number of days Bob and Carol visit together is $N_{BC} = \lfloor \frac{m}{\text{lcm}(b,c)} \rfloor$.
- The number of days Carol and Alice visit together is $N_{CA} = \lfloor \frac{m}{\text{lcm}(c,a)} \rfloor$.
- The number of days all three visit together is $N_{ABC} = \lfloor \frac{m}{\text{lcm}(a,b,c)} \rfloor$.

Since exactly 6 total points are distributed on any day where at least one person visits:

- If exactly 1 person visits, they get 6 points.
- If exactly 2 people visit, they each get 3 points.
- If exactly 3 people visit, they each get 2 points.

For Alice, we can calculate her total points by summing the points she receives from all days she visits:

- Total days Alice visits is N_A, which would initially give her $6 \times N_A$ points (assuming she was alone).
- But on days she visits with Bob (N_{AB}), they share the points (she only gets 3 instead of 6, a deduction of 3). Thus we subtract $3 \times N_{AB}$.
- Similarly, on days she visits with Carol (N_{CA}), we subtract $3 \times N_{CA}$.
- On days all three visit (N_{ABC}), we deducted 3 for Bob and 3 for Carol, effectively reducing her points by 6 (from 6 to 0). But she actually receives 2 points. Thus, we must add back $2 \times N_{ABC}$.

The final formula for Alice's points is:

$$P_A = 6N_A - 3N_{AB} - 3N_{CA} + 2N_{ABC}$$

Symmetrically, for Bob and Carol:

$$P_B = 6N_B - 3N_{AB} - 3N_{BC} + 2N_{ABC}$$

$$P_C = 6N_C - 3N_{BC} - 3N_{CA} + 2N_{ABC}$$

Important Considerations

Since $m \leq 10^{17}$, the LCM of three numbers up to 10^6 can exceed 10^{18}. However, if the LCM exceeds m, the division $\lfloor \frac{m}{\text{lcm}} \rfloor$ will simply be 0. We can safely cap the LCM calculation at $m+1$ to prevent any `long long` overflow during intermediate steps. We check if `(a / gcd) * b > m` before performing the multiplication.

Time Complexity

- **Time Complexity:** $\mathcal{O}(\log(\min(a, b, c)))$ per testcase to compute the GCD/LCM.
- **Space/Memory Complexity:** $\mathcal{O}(1)$.

Source Code (C++)

```
#include <iostream>

using namespace std;

long long gcd(long long a, long long b) {
    while (b) {
        a %= b;
        swap(a, b);
    }
    return a;
}

long long get_lcm(long long a, long long b, long long m) {
    if (a == 0 || b == 0) return m + 1;
    long long g = gcd(a, b);
    long long part = a / g;
    if (m / part < b) return m + 1;
    long long lcm = part * b;
    if (lcm > m) return m + 1;
    return lcm;
}

void solve() {
    long long a, b, c, m;
    cin >> a >> b >> c >> m;

    long long n_a = m / a;
    long long n_b = m / b;
    long long n_c = m / c;

    long long lcm_ab = get_lcm(a, b, m);
    long long lcm_bc = get_lcm(b, c, m);
    long long lcm_ca = get_lcm(c, a, m);
    long long lcm_abc = get_lcm(lcm_ab, c, m);

    long long n_ab = m / lcm_ab;
    long long n_bc = m / lcm_bc;
    long long n_ca = m / lcm_ca;
    long long n_abc = m / lcm_abc;
```

```
    long long p_a = 6 * n_a - 3 * n_ab - 3 * n_ca + 2 * n_abc;
    long long p_b = 6 * n_b - 3 * n_ab - 3 * n_bc + 2 * n_abc;
    long long p_c = 6 * n_c - 3 * n_bc - 3 * n_ca + 2 * n_abc;

    cout << p_a << " " << p_b << " " << p_c << "\n";
}

int main() {
    ios_base::sync_with_stdio(false);
    cin.tie(NULL);

    int t;
    if (cin >> t) {
        while (t--) {
            solve();
        }
    }
    return 0;
}
```